



Report

Project 26: Cigarette Smoker Problem with Synchronisation

CSC-204 Operating Systems

Name	Enrollment No.
Arnav Gupta	23114010
Krish Singla	23114050
Megh Bhavesh Shah	23114065
Vaghasiya Jay Rasikbhai	23114104
Tanay Kapadia	23323044

April 27, 2025

Contents

1	Introduction	3
2	Background	3
2.1	Naive Solution	4
3	Proposed Solutions	5
3.1	Solution using a Novel Approach	6
3.1.1	Proof of Correctness	8
3.1.2	Expected Number of Steps for Smoking a Cigarette	9
3.2	Solution using Semaphore Arrays and Bitmasking	11
3.3	Solution using Monitors	13
4	Performance Analysis and Comparison of Solutions	15
4.1	Memory Usage and Page Faults	15
4.2	Number of Smoking Rounds per Second	17
4.3	Detailed Description of Threads	19
5	Conclusion	21
5.1	Challenges Faced	22

1 Introduction

With the emergence of multi-processor systems, tasks need to be managed efficiently. To manage the resources efficiently, synchronization is needed. Synchronization ensures perfect execution of processes and controlled access to shared resources. The cigarette-smoker problem is an example of a synchronization problem in operating systems.

The problem statement of the cigarette smokers' problem is as follows:

Assume that a cigarette can be made by three ingredients: tobacco, paper and matches. There are three smokers and one agent. Smoker 1 has unlimited supply of tobacco, Smoker 2 has unlimited supply of paper and Smoker 3 has unlimited supply of matches. The agent has unlimited supply of all the three ingredients.

The agent will place any two of the three ingredients on the table. The smoker who has the third supply should remove the two items from the table, using them (along with their own supply) to make a cigarette, which they smoke for a while. Once the smoker has finished his cigarette, the agent places two new random items on the table. This process continues forever.

To implement this situation, we will use three different semaphores for the ingredients. The agent function will signal two of the three semaphores randomly, and the smoker function (for each smoker) will wait for the two semaphores which are not available to him.

However, this can lead to deadlock; if the agent places paper and tobacco on the table, the smoker with tobacco may remove the paper and the smoker with matches may take the tobacco, leaving both unable to make their cigarette. The solution is to define additional processes and semaphores that prevent deadlock, without modifying the agent.

2 Background

Suhas Patil, an Indian-American academic and computer scientist, proposed the cigarette smokers problem for the first time.

Patil also imposed two restrictions on the solution of this problem:

1. The agent code is not modifiable.
2. The solution is not allowed to use conditional statements.

He justified the conditions, because if the agent represents an operating system, it would be unreasonable or impossible to modify it every time a new application came along. In Patil's description of the problem, the agent is considered to be an unchangeable entity with fixed functionalities.

The actions of the agent can be summarised with code 1.

```

1 void agent() {
2     while(true) {
3         sem_wait(&next_round);
4
5         int number = rand() % 3;
6         switch(number) {
7             case 0:
8                 sem_post(&paper);
9                 sem_post(&match);
10                break;
11             case 1:
12                 sem_post(&match);
13                 sem_post(&tobacco);
14                break;
15             case 2:
16                 sem_post(&tobacco);
17                 sem_post(&paper);
18                break;
19         }
20     }
21 }

```

Listing 1: Agent Function

The semaphore `next_round` denotes that the agent starts the next round by putting two of the three ingredients randomly, and the semaphores `tobacco`, `paper` and `match` are for their corresponding semaphores.

The agent randomly selects any one of the three ingredients which is not to be placed on the table, and for each case, signals the two semaphores corresponding to the other two ingredients.

2.1 Naive Solution

For every smoker, we can define a function in which the smoker waits for the two ingredients that it does not have. After getting the signal from both the ingredients, the smoker will continue to make the cigarette and smoke it, after which it will signal the `next_round` semaphore denoting that they completed smoking their cigarette.

```

1 void smoker_with_tobacco() {
2     while (true) {
3         sem_wait(&paper);
4         sem_wait(&match);
5         // at this point, both paper and match has been received
6
7         // finished smoking
8         sem_post(&next_round);
9     }
10 }

```

Listing 2: Naive Solution

Code 2 corresponds to the smoker with tobacco. It will wait for the other two ingredients, paper and match. After the signals are fetched by the wait statements, the smoker starts making the cigarette and smoking, and then signals the semaphore `next_round`. Similar functions can be written for the other two smokers.

Suppose the agent places match and tobacco on the table. Ideally, the smoker with paper should grab the two items and make and smoke the cigarette. But, there can be a case where the signal for match is fetched by the wait statement by the `smoker_with_paper` and the signal for tobacco is fetched by the `smoker_with_match`. Here, both the smokers acquired an ingredient, but the requirement of any of the smokers is not completely fulfilled and none of the smokers can continue further. Hence, a **deadlock** occurred, as seen in Table 1.

<code>smoker_with_tobacco</code>	<code>smoker_with_match</code>	<code>smoker_with_paper</code>
<code>sem_wait(&paper)</code> BLOCKED	<code>sem_wait(&tobacco)</code> SUCCESS	<code>sem_wait(&match)</code> SUCCESS
<code>sem_wait(&match)</code> BLOCKED	<code>sem_wait(&paper)</code> BLOCKED	<code>sem_wait(&tobacco)</code> BLOCKED

Table 1: Possible scenario when agent drops tobacco and match

It can be shown that any ordering of the `wait` statements in the smoker functions would not ensure the complete removal of deadlock. There will always be a case, where in a specific order of ingredients signalled, a deadlock will occur. Hence, the naive solution is not correct.

The problem is stated so as to allow the definition of any number of semaphores and functions and to use any number of `wait` and `signal` statements to write programs for the smoker users which will use the resources and be free of deadlocks. Conditional statements are not allowed in the program.

Patil states that this problem has no solution.

Patil does so by introducing Petri nets as a representation of a process by highlighting the flow of control in the process. Petri nets are directed graphs with two types of nodes called places and transitions. The places have an ability to hold a *marker* and a transition is said to be enabled when all input places of the transition have markers.

The proof proceeds by defining a Petri net representation of any solution, if it exists. Petri nets in which no more than one input place of a transition is shared as input place with other transitions are called *simple* Petri nets. He claims that for any process, if no conditional statements are used, the Petri net representation of the process is (or can be converted into) a simple Petri net. He proved mathematically that Figure 1 cannot be converted into a simple Petri net.

3 Proposed Solutions

We have proposed three major solutions for the cigarette smokers problem. These solutions are based on different synchronisation techniques, and are done

1. by using semaphore with an additional operation,
2. by using semaphore arrays and bitmasking,
3. and by using monitors

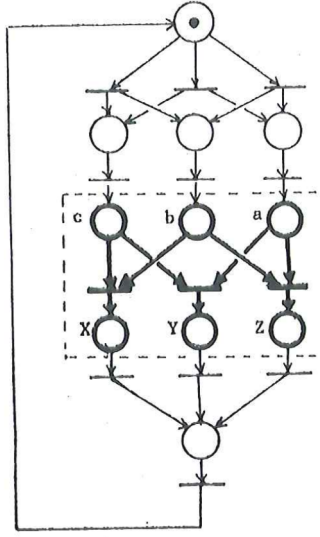


Figure 1: Petri net representing the desired configuration

We have also proposed solutions for a more **generalized version** of this problem. This problem statement is as follows:

Suppose we have n smokers and n ingredients. We also have an agent who has an infinite supply of the n ingredients. Each smoker has one ingredient. The ingredients possessed by all the n smokers are distinct. Further, each smoker requires all the n ingredients for smoking. The agent now drops $n - 1$ ingredients. The smokers will try to grab all the $n - 1$ ingredients. The smoker who gets all the n ingredients will smoke and the cycle continues with the agent dropping the next round of ingredients.

3.1 Solution using a Novel Approach

The key claim of the cigarette smokers problem is that the problem has no solution for traditional semaphores, as they existed at the time. When this problem was initially proposed by Patil, semaphores only provided operations for incrementing or decrementing their internal value by one (i.e, `wait` and `signal`). We have already seen that if we are limited to these two operations only, there are situations in which avoiding deadlock is provably impossible. Regardless of how the agent and the smoker threads are constructed, once the agent's structure is fixed, any construction of the smokers will create a possible deadlock situation.

Here, we propose another solution by defining a new operation of the semaphore. We have created our very own implementation of a mutex lock and a semaphore built using that mutex lock. The implementation can be seen in Code 3. This is made possible using the `std::atomic` library provided in C++, which provides many data types of atomic nature. However, we were only concerned with the atomic lock and the `test_and_wait` and `compare_and_swap` functions. Our custom implementation of the semaphore includes the standard operations such as `init`, `wait`, `signal`. Moreover, we have included a new function known as `try_wait`. The newly added function makes the solution possible.

```

1  typedef struct {
2      int value;
3      Mutex mutex;
4  } BinarySemaphore;
5
6  void binary_semaphore_init(BinarySemaphore *sem, int value) {
7      if (value != 0 && value != 1) {
8          fprintf(stderr, "Error: Binary semaphore must be initialized
9              to 0 or 1.\n");
10             return;
11         }
12         sem->value = value;
13         mutex_init(&sem->mutex);
14     }
15
16 void binary_semaphore_wait(BinarySemaphore *sem) {
17     mutex_lock(&sem->mutex);
18     while (sem->value == 0) {
19         mutex_unlock(&sem->mutex);
20         mutex_lock(&sem->mutex);
21     }
22     sem->value = 0;
23     mutex_unlock(&sem->mutex);
24 }
25
26 void binary_semaphore_signal(BinarySemaphore *sem) {
27     mutex_lock(&sem->mutex);
28     sem->value = 1;
29     mutex_unlock(&sem->mutex);
30 }
31
32 int binary_semaphore_try_wait(BinarySemaphore *sem) {
33     mutex_lock(&sem->mutex);
34     if (sem->value == 1) {
35         sem->value = 0;
36         mutex_unlock(&sem->mutex);
37         return 0;
38     } else {
39         mutex_unlock(&sem->mutex);
40         return -1;
41     }
42 }
43
44 void binary_semaphore_destroy(BinarySemaphore *sem) {
45     mutex_destroy(&sem->mutex);
46 }

```

Listing 3: Semaphore Implementation

Let us analyze the `try_wait()` operation. This function tries to get the value of semaphore and only locks the semaphore if it is not currently locked, i.e. when the value of the semaphore is non-zero. If the value of semaphore is found to be zero, then it safely exits the functions without locking the semaphore. This means that there is no busy waiting for the value of semaphore to return to positive.

The algorithm of the solution then becomes that the smoker with, say, tobacco waits for paper. If he gets access to paper, he is trying for matches. If he gets access to the match, then he has received both the resources he needed and thus, the cigarette is rolled. If the access to the match is not received, then he will also drop the paper he had in the first place. Notice that the last step is made possible through our custom semaphore.

The correctness of this algorithm may not be obvious as the process is not deterministic anymore. This is discussed mathematically. Also, this algorithm was for the classical problem with 3 smokers, and it may not be trivial to generalize it further.

3.1.1 Proof of Correctness

Though we have seen that deadlock can never occur when we're using `sem_try_wait()`, we can still experience **livelock**. Livelock is a situation where the threads are not blocked, but they are not making any progress either. Let us understand this with an example.

Suppose the agent drops paper and match on the table. The smoker with tobacco will try to get the paper first and will be successful in doing so. But before he can get the match, the smoker with paper would have already picked up the match. Now, both the smokers are not able to make the cigarette. Thus, they both drop their acquired resources one after the other. So, we can imagine a situation where the two smokers are repeatedly dropping and picking up the same resources in the hope of getting the other resource. This is a situation of livelock.

Now, let us probabilistically analyze this situation and determine the probability of the correct smoker getting both ingredients.

First, without loss of generality, assume that the agent drops paper and match. Each smoker will have equal probability of getting the ingredient that the smoker does not have. Let us define three states A , B and C as follows:

- A : The smoker with tobacco gets both the paper and match.
- B : The smoker with tobacco gets either the paper or match, but not both.
- C : The smoker with tobacco gets neither the paper nor match.

Thus, we need to find $\mathbb{P}(A)$ the probability of the event A occurring. Since each resource is demanded by two smokers with equal likelihood, and only one of the two smokers is supposed to get the resource, we can say that the probability of the person getting the resource is $\frac{1}{2}$. Hence, we get:

$$\mathbb{P}(A) = \frac{1}{2} \cdot \frac{1}{2} + \binom{2}{1} \cdot \frac{1}{2} \cdot \frac{1}{2} \cdot \mathbb{P}(A | B) + \frac{1}{2} \cdot \frac{1}{2} \cdot \mathbb{P}(A | C) \quad (1)$$

Now, let us calculate $\mathbb{P}(A | B)$, the probability of reaching state A from state B . When the system is in state B , one of the two smokers holding resources will drop the resource with equal probability, and that resource will either be acquired by the same person or the other person waiting for it, again with equal probability.

$$\mathbb{P}(A \mid B) = \frac{1}{2} \cdot \left[\frac{1}{2} \cdot \mathbb{P}(A \mid A) + \frac{1}{2} \cdot \mathbb{P}(A \mid B) \right] + \frac{1}{2} \cdot \left[\frac{1}{2} \cdot \mathbb{P}(A \mid B) + \frac{1}{2} \cdot \mathbb{P}(A \mid C) \right] \quad (2)$$

Putting $\mathbb{P}(A \mid A) = 1$ in equation 2, the RHS of equations 1 and 2 become equivalent to each other. Hence, we conclude that $\mathbb{P}(A \mid B) = \mathbb{P}(A)$. Putting this in equation 1, we get:

$$2 \cdot \mathbb{P}(A) - 1 = \mathbb{P}(A \mid C) \quad (3)$$

Now, in the same way, let us calculate $\mathbb{P}(A \mid C)$.

$$\mathbb{P}(A \mid C) = \frac{1}{2} \cdot [\mathbb{P}(A \mid B) + \mathbb{P}(A \mid C)] \quad (4)$$

Putting $\mathbb{P}(A \mid B) = \mathbb{P}(A)$ and $\mathbb{P}(A \mid C) = 2 \cdot \mathbb{P}(A) - 1$ in equation 4, we get $\boxed{\mathbb{P}(\mathbf{A}) = 1}$.

This means that no matter how many times the smokers drop and pick up the resources, they will eventually get the required resources. Now, let us try to analyze the expected number of steps it will require to reach the desired state.

3.1.2 Expected Number of Steps for Smoking a Cigarette

There are three persons **A**, **B**, and **C**, and three types of resources **x**, **y**, and **z**. The shorthand initial configuration is $((\mathbf{x}), (\mathbf{y}), (\mathbf{z}))$, where:

- The first person has infinite resources of type **x**,
- The second person has infinite resources of type **y**,
- The third person has infinite resources of type **z**.

We aim to find **P**, the expected number of steps so that a smoker can smoke a cigarette. Without loss of generality, we assume that **x** and **y** are put on the table. And also we are assuming that in one clock tick, only one operation can happen that means either a person can pick up a resource or drop a resource.

We define six possible scenarios, and **P_i** are the expected number of steps for each scenario for a smoker to smoke a cigarette with the following initial configurations:

$$\begin{aligned} \mathbf{P}_1 &: ((\mathbf{x}), (\mathbf{y}), (\mathbf{z})), \\ \mathbf{P}_2 &: ((\mathbf{x}), (\mathbf{y}), (\mathbf{x}, \mathbf{z})), \\ \mathbf{P}_3 &: ((\mathbf{x}, \mathbf{y}), (\mathbf{y}), (\mathbf{x}, \mathbf{z})), \\ \mathbf{P}_4 &: ((\mathbf{x}), (\mathbf{x}, \mathbf{y}), (\mathbf{z})), \\ \mathbf{P}_5 &: ((\mathbf{x}, \mathbf{y}), (\mathbf{x}, \mathbf{y}), (\mathbf{z})), \\ \mathbf{P}_6 &: ((\mathbf{x}), (\mathbf{x}, \mathbf{y}), (\mathbf{y}, \mathbf{z})). \end{aligned}$$

Finding P_1

$$P_1 = \frac{1}{2} \left[1 + \frac{1}{2}(1) + \frac{1}{2}(1 + P_3) \right] + \frac{1}{2} \left[1 + \frac{1}{2}(1 + P_5) + \frac{1}{2}(1 + P_6) \right].$$

- First parenthesis is **C** picks up **x** (or **y** — symmetrical case, so doesn't matter), second parenthesis is **A** picks up **y** (or **B** picks up **x** — symmetrical case, so doesn't matter) and both are having equal probability of $\frac{1}{2}$.
- In the first parenthesis, the first term is one step for **C** to pick up **x**, the second term is one step for **C** to pick up **y** with probability $\frac{1}{2}$ and the third term is one step for **A** to pick up **y** with probability $\frac{1}{2}$, leading to **P₃**.
- In the second parenthesis, the first term is one step for **B** to pick up **x** (or **A** picks up **y** — symmetrical case, so doesn't matter), the second term is one step for **A** to pick up **y** with probability $\frac{1}{2}$ and the third term is one step for **C** to pick up **y** with probability $\frac{1}{2}$, leading to **P₅**.

Finding P_3

$$P_3 = \frac{1}{2} \left[1 + \frac{1}{2}(1) + \frac{1}{2}(1 + P_3) \right] + \frac{1}{2} \left[1 + \frac{1}{2}(1 + P_3) + \frac{1}{2}(1 + P_5) \right].$$

- Here first parenthesis is **A** drops **y** first with probability $\frac{1}{2}$ and second parenthesis is **C** drops **x** first with probability $\frac{1}{2}$.
- In the first parenthesis, the first term is one step for **A** to drop **y**, the second term is one step for **C** to pick up **x** with probability $\frac{1}{2}$ and the third term is one step for **A** to pick up **y** with probability $\frac{1}{2}$, leading to **P₃**.
- In the second parenthesis, the first term is one step for **C** to drop **x**, the second term is one step for **C** to pick up **x** again with probability $\frac{1}{2}$ leading to **P₃** again and the third term is one step for **B** to pick up **x** with probability $\frac{1}{2}$ leading to **P₅**.

Finding P_6

$$P_6 = \frac{1}{2} \left[1 + \frac{1}{2}(1) + \frac{1}{2}(1 + P_6) \right] + \frac{1}{2} \left[1 + \frac{1}{2}(1 + P_6) + \frac{1}{2}(1 + P_5) \right].$$

- Here first parenthesis is **B** drops **x** first with probability $\frac{1}{2}$ and second parenthesis is **C** drops **y** first with probability $\frac{1}{2}$.
- In the first parenthesis, the first term is one step for **B** to drop **x**, the second term is one step for **C** to pick up **y** with probability $\frac{1}{2}$ and the third term is one step for **B** to pick up **x** with probability $\frac{1}{2}$, leading to **P₆**.

- In the second parenthesis, the first term is one step for **C** to drop **y**, the second term is one step for **C** to pick up **y** again with probability $\frac{1}{2}$ leading to **P₆** again and the third term is one step for **A** to pick up **y** with probability $\frac{1}{2}$ leading to **P₅**.

Finding **P₅**

$$\mathbf{P}_5 = \frac{1}{2} \left[1 + \frac{1}{2}(1 + \mathbf{P}_5) + \frac{1}{2}(1 + \mathbf{P}_6) \right] + \frac{1}{2} \left[1 + \frac{1}{2}(1 + \mathbf{P}_5) + \frac{1}{2}(1 + \mathbf{P}_6) \right].$$

- Here both cases are symmetrical, **A** drops **y** first with probability $\frac{1}{2}$ and **B** drops **x** first with probability $\frac{1}{2}$.
- In the first parenthesis, the first term is one step for **A** to drop **y**, the second term is one step for **A** to pick up **y** with probability $\frac{1}{2}$ leading to **P₅** again and the third term is one step for **C** to pick up **x** with probability $\frac{1}{2}$ leading to **P₆**.
- In the second parenthesis, the first term is one step for **B** to drop **x**, the second term is one step for **B** to pick up **x** with probability $\frac{1}{2}$ leading to **P₅** again and the third term is one step for **C** to pick up **y** with probability $\frac{1}{2}$ leading to **P₆**.

Conclusion

The equations for **P₁**, **P₃**, **P₅**, and **P₆** can be solved iteratively or using a system of linear equations to find the expected number of steps for a smoker to smoke a cigarette.

The final answer is 8.75 steps.

3.2 Solution using Semaphore Arrays and Bitmasking

This solution precisely solves the cigarette smoker problem by not violating any of the restrictions posed and using the same semaphore primitive described. Moreover, this solution is made for the generalised problem statement. In fact, it directly contradicts Patil's claim.

We need to synchronize the actions of the N smokers such that a deadlock should not occur. For that, we will use a variable t to denote the configuration of the ingredients on the table.

Suppose, there is an N -bit number t , where each bit of the number corresponds to each of the n ingredients. Whenever an ingredient is placed on the table, the bit corresponding to that ingredient will be set. So, the final value of the number t will be such that $(N - 1)$ bits out of the N bits will be set, denoting that $(N - 1)$ ingredients are placed on the table.

Now, each smoker should respond to that value of t in which the bit corresponding to that ingredient which is available to that smoker is not set. For example, the smoker that has the ingredient corresponding to the last bit should respond to the value of $t = (2^N - 1) - 1$, and so on.

Therefore, the suggested algorithm using bit-masks is as follows:

The agent will release the signals of $(N - 1)$ out of N ingredients randomly. We have to set the corresponding $(N - 1)$ bits in the number t , initialised 0 at first. Therefore we

will make N functions (vector of functions) corresponding to the N ingredients. In each function, we have to set the bit of that ingredient in t . So, in the i^{th} function (of the i^{th} ingredient) we will add 2^i to t . We will then call the $(N - 1)$ functions of the ingredients placed on the table by the agent. Now, the smoker with the i^{th} ingredient should wait for the value of $t = 2^N - 1 - 2^i$.

To implement this, we will make an array S of 2^N semaphores. Under each ingredient function, after adding 2^i to t , we will signal $S[t]$. Each smoker function will wait for $S[t]$, where $t = 2^N - 1 - 2^i$, and i = the bit corresponding to the ingredient that the smoker has. To set the $(N - 1)$ bits **one-by-one**, we will define a mutex lock. The ingredient functions will wait for mutex, set the bit of the ingredient and then signal the mutex.

The implementation is shown in Code 4 and the smoker functions are in Code 5. Note that they make use of our implementation of mutex and semaphore.

```

1 void* f(void* arg) {
2     int id = *(int*)arg;
3
4     while (1) {
5         binary_semaphore_wait(&ingredients[id]);
6
7         mutex_lock(&t_mutex);
8         t = t + (1 << id);
9         binary_semaphore_signal(&targets[t]);
10        mutex_unlock(&t_mutex);
11    }
12 }
```

Listing 4: Semaphore Array Logic

```

1 void* smoker(void* arg) {
2     int id = *(int*)arg;
3
4     while (1) {
5         binary_semaphore_wait(&targets[(1 << N) - 1 - (1 << id)]);
6
7         mutex_lock(&t_mutex);
8         t = 0;
9         mutex_unlock(&t_mutex);
10        binary_semaphore_signal(&next_round);
11    }
12 }
```

Listing 5: Smoker Functions

The mutex lock ensures that the variable t is modified only by one function at a time. This solution ensures that there are no conditional statements used, as we have assigned a number for each configuration. This problem uses memory of size $\Theta(2^N)$, as we have assigned N bits for each ingredient.

3.3 Solution using Monitors

Semaphores are powerful but low-level data structures. A programmer needs to initialise and manage when a semaphore will wait or signal. Since humans are prone to mistakes, it is really easy for any programmer to mess up the order or forget one wait or signal statement. One single mistake can ruin the flow of the whole synchronisation process. Monitors are a **higher-level abstraction**. Instead of the programmer manually tracking the wait/signal statements of a semaphore, the monitor does it automatically. When a function is called, the monitor automatically locks access to the other functions. Hence, monitors are generally preferred over semaphores.

We also have implemented the solution for N smokers using monitors. Under the monitor class, we have defined the functions `dropIngredients`, `pickUpIngredients` and `notifyForNextRound`, because at one point of time, either the agent will drop the ingredients, the smoker will pick up the ingredients or the agent will be notified to continue with the next round.

The algorithm we used is as follows:

Firstly, the agent will choose an ingredient out of the N ingredients to “not drop” on the table. All the other $N - 1$ ingredients will be dropped on the table. Therefore, the other ingredients are collectively sent to the `dropIngredients` function.

```
1 void agent() {
2     while (true) {
3         int exclude = rand() % N;
4
5         array<int, N - 1> items;
6         int index = 0;
7         for (int j = 0; j < N; ++j) {
8             if (j != exclude) {
9                 items[index++] = j;
10            }
11        }
12
13        csp.dropIngredients(items);
14    }
15 }
```

Listing 6: Agent Function

A monitor is a module wrapped around a mutex and also has to use condition variables on which the operations must always be thread-safe. Therefore it makes sense to implement a monitor using a C++ class. Code 7 shows the private members of the class.

```
1 class CigaretteSmokingProblem {
2
3 private:
4     mutex mtx;
5     condition_variable next_round;
6     array<condition_variable, N> smokers;
7     int current_smoker = -1;
8 };
```

Listing 7: Monitor Class Definition

The `dropIngredients` function logs which ingredients are dropped and then notifies that smoker who contains the missing ingredient. It will signal the conditional variable corresponding to the smoker. The `pickupIngredients` function, which requires an id to identify the smoker, contains a conditional variable which waits for all those smokers that are not currently smoking. The smoker, after completion, calls the `notifyForNextRound` that signals the `next_round` conditional variable, notifying the agent to continue with the next round.

```

1 void dropIngredients(array<int, N - 1> ingredients) {
2     unique_lock<mutex> lock(mtx);
3
4     next_round.wait(lock, [this] { return current_smoker ==
5         -1; });
6
7     int missing = 0;
8     for (int i = 0; i < N; ++i)
9         missing ^= i;
10    for (auto i : ingredients)
11        missing ^= i;
12
13    current_smoker = missing;
14    smokers[missing].notify_one();
15 }
16
17 void pickupIngredients(int id) {
18     unique_lock<mutex> lock(mtx);
19     smokers[id].wait(lock, [this, id] { return
20         current_smoker == id; });
21 }
22
23 void notifyForNextRound() {
24     unique_lock<mutex> lock(mtx);
25     current_smoker = -1;
26     next_round.notify_one();
27 }

```

Listing 8: Monitor Member Functions

The mutex lock `mtx` ensures that two threads do not access a particular function at the same time. The proposed solution with monitors is correct, because there is synchronization between the dropping of the ingredients by the agent, and picking up of the ingredients by the corresponding smoker. A deadlock cannot occur because other smokers apart from the concerned smoker would wait until their turn for smoking does not occur.

4 Performance Analysis and Comparison of Solutions

We have implemented the solutions of the Cigarette Smoking Problems using both the semaphores and monitors. As mentioned before, the monitors are generally used to reduce the chances of ruining the workflow and synchronization through manual error. We will now compare the solutions of this problem using semaphores and monitors based on many different metrics.

4.1 Memory Usage and Page Faults

In the above mentioned solutions, we have seen that when we implement using semaphores, we use memory space of the size $\Theta(2^N)$, where N is the number of smokers and ingredients. This was because we were assigning a number to each configuration of the ingredients. While, in the solution using monitors, we use memory of size $\Theta(N)$, which is significantly smaller than $\Theta(2^N)$. Thus, in terms of memory used, the monitor solution is more efficient than the semaphore solution.

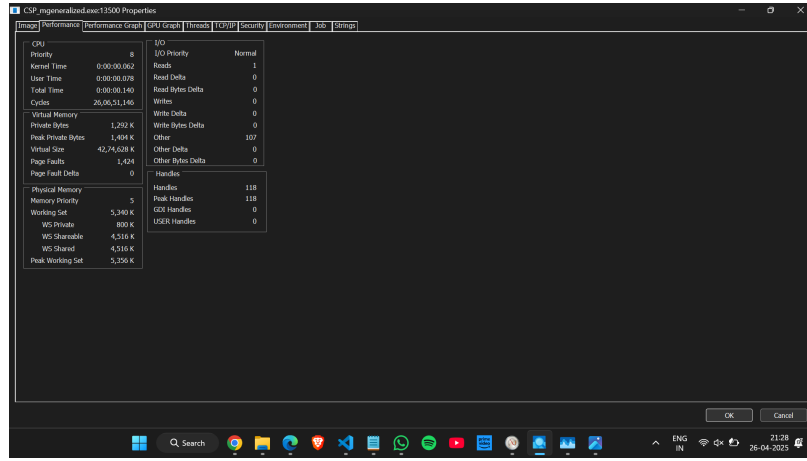


Figure 2: Memory Usage of the Monitor Solution

We ran both the solutions for $N = 10$. The solution using monitors should use memory around 10 KB, while that using semaphores, memory of around 1000 KB should be used. Therefore, a factor of approximately 100 is expected to differ in the solutions. As evident from the data (attached below), we see that monitors used 150 KB and semaphores used around 2,00,000 KB memory space.

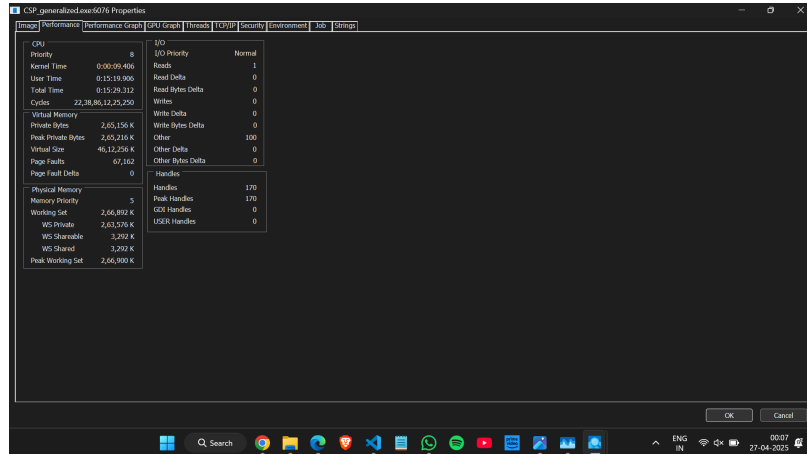
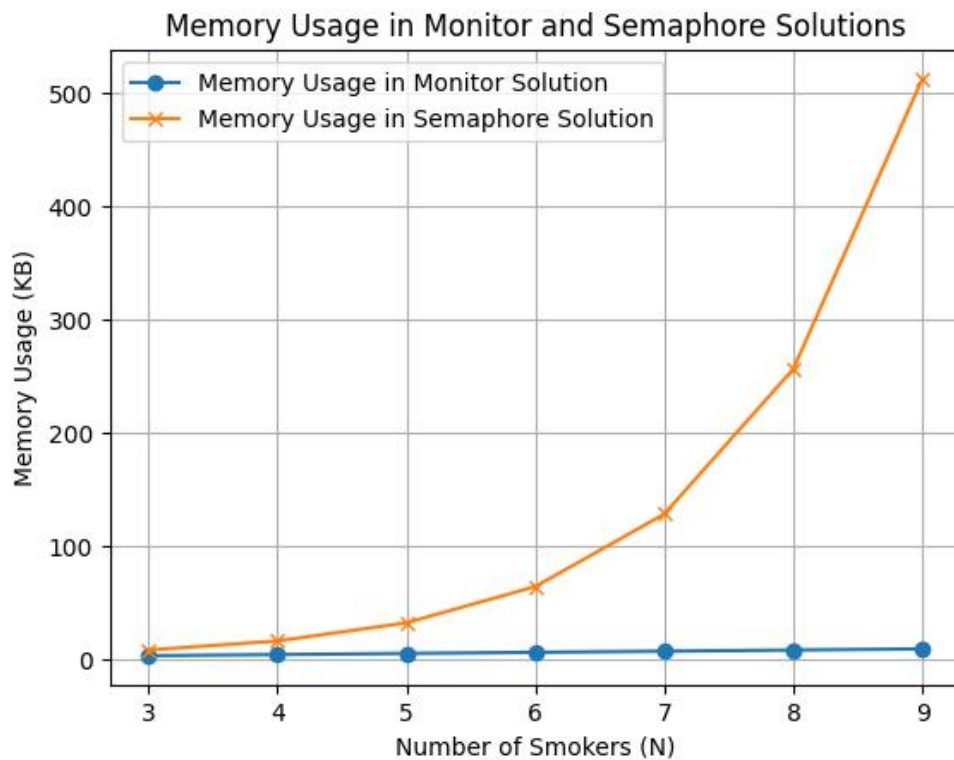


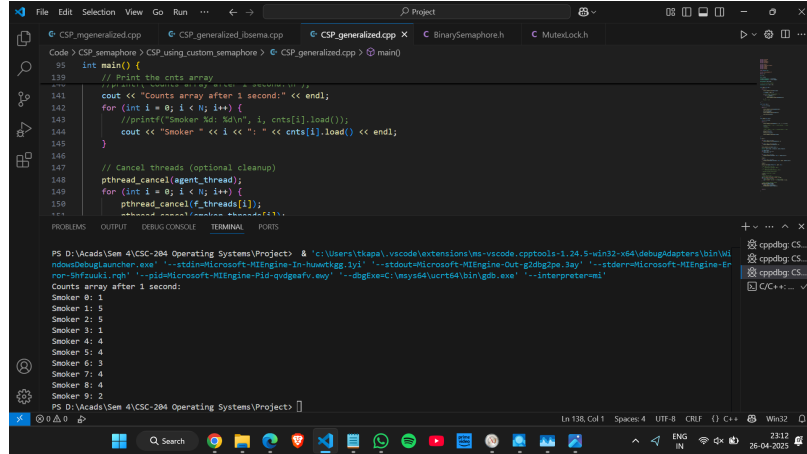
Figure 3: Memory Usage of the Semaphore Solution

As shown in the images, the virtual memory allotted to both the programs is almost the same (42 lac KB). But the memory used and the working set in the semaphores' solution (2.5 lac KB) compared to the monitors' solution (5000 KB), the page faults in the former solution (70,000) is much higher than the later one (1500).



4.2 Number of Smoking Rounds per Second

We ran the solution using semaphores using the *wait and hold* function of the semaphore, which meant that the thread would still utilise the CPU until its requirements were met, theoretically reducing the throughput to an extent. When we ran this solution for 10 seconds, 40 rounds of smoking were completed. Therefore the throughput came out to be 4 rounds per second, which is very low.

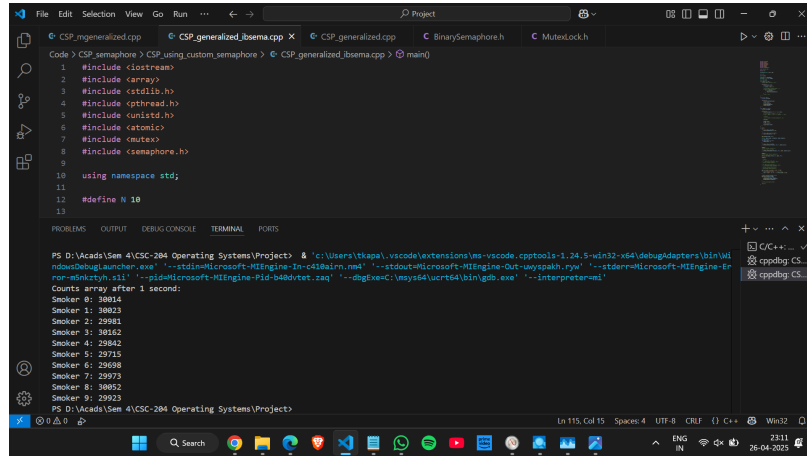


```
Code > CSP_semaphore > CSP_using_custom_semaphore > CSP_generalized.cpp > main()
95 int main() {
139 // Print the counts array
140 // Print the counts array after 1 second:
141 cout << "Counts array after 1 second:" << endl;
142 for (int i = 0; i < N; i++) {
143     // Print the counts array after 1 second:
144     cout << "Smoker " << i << ": " << cnts[i].load() << endl;
145 }
146
147 // Cancel threads (optional cleanup)
148 pthread_cancel(agent_thread);
149 for (int i = 0; i < N; i++) {
150     pthread_cancel(t_threads[i]);
151 }
152 }
```

PS D:\Acad\Sem 4\CSC-204 Operating Systems\Project> .\CSP_generalized.exe
Counts array after 1 second:
Smoker 0: 1
Smoker 1: 5
Smoker 2: 5
Smoker 3: 1
Smoker 4: 4
Smoker 5: 4
Smoker 6: 3
Smoker 7: 4
Smoker 8: 4
Smoker 9: 2

Figure 4: Throughput of the Semaphore Solution when *wait and hold* is used

Then, we modified the solution using the in-built features of semaphores. When we ran this solution for 10 seconds, approximately 3,00,000 smoking rounds were completed (as shown in the figure). Hence, the throughput came out to be 30000 rounds per second.



```
Code > CSP_semaphore > CSP_using_custom_semaphore > CSP_generalized.cpp > main()
1 #include <iostream>
2 #include <array>
3 #include <mutex>
4 #include <thread>
5 #include <unistd.h>
6 #include <atomic>
7 #include <mutex>
8 #include <semaphore.h>
9
10 using namespace std;
11
12 #define N 10
13
14 // ... (other code) ...
15
16 // Print the counts array after 1 second:
17 cout << "Counts array after 1 second:" << endl;
18 for (int i = 0; i < N; i++) {
19     // Print the counts array after 1 second:
20     cout << "Smoker " << i << ": " << cnts[i].load() << endl;
21 }
```

PS D:\Acad\Sem 4\CSC-204 Operating Systems\Project> .\CSP_generalized.exe
Counts array after 1 second:
Smoker 0: 9999
Smoker 1: 9999
Smoker 2: 9999
Smoker 3: 9999
Smoker 4: 9999
Smoker 5: 9999
Smoker 6: 9999
Smoker 7: 9999
Smoker 8: 9999
Smoker 9: 9999

Figure 5: Throughput of the Semaphore Solution with in-built functions

4.3 Detailed Description of Threads

For $N = 10$ smokers, the number of threads formed in the semaphore solutions are 22: 1 for each smoker, 1 for each intermediate function, 1 for the agent and 1 for the main function. The number of threads formed in the monitor solution are 12: 1 for each smoker, 1 for the agent and 1 for the main function.

Out of the 22 threads in the semaphore solution, there are four types of threads, the agent thread, the intermediate thread, the smoker thread and the main thread. The approximate number of context switches observed in the three types of threads are:

Type of Thread	Number of Context Switches (approx.)
Smoker Thread	100
Intermediate Thread	1000
Agent Thread	1000
Main Thread	10000

Table 2: Context Switches for Different Threads

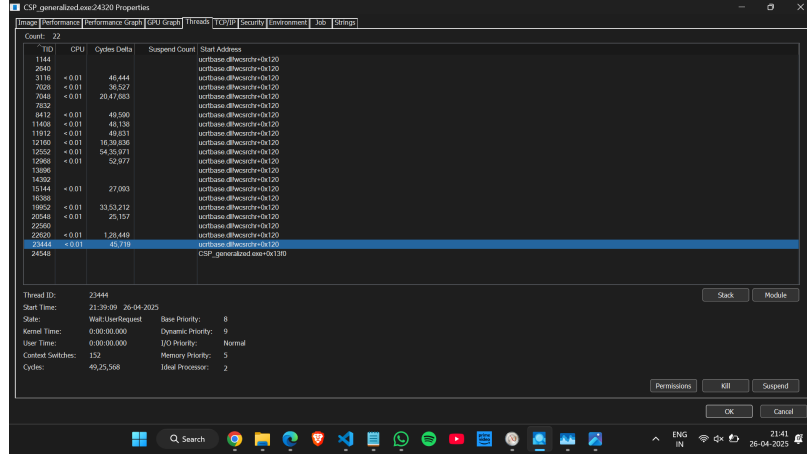


Figure 7: Context Switches of Smoker Thread in the Semaphore Solution

This can be explained as follows: Every time the agent thread is called, it drops the nine ingredients, therefore invoking the nine intermediate functions. Thus, each time the agent thread switches context, the intermediate threads also switch their context. But, probabilistically each smoker thread is invoked for $\frac{1}{10}^{th}$ times as that of the agent thread. Hence, the agent thread and each intermediate thread's context switches are almost identical (~ 1000) and each smoker thread's context switches are $\frac{1}{10}^{th}$ of the agent thread (~ 100).

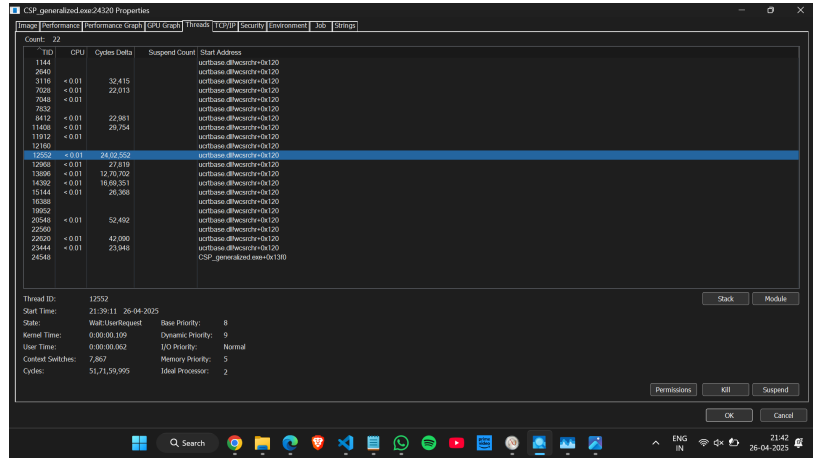


Figure 8: Context Switches of Agent Thread in the Semaphore Solution

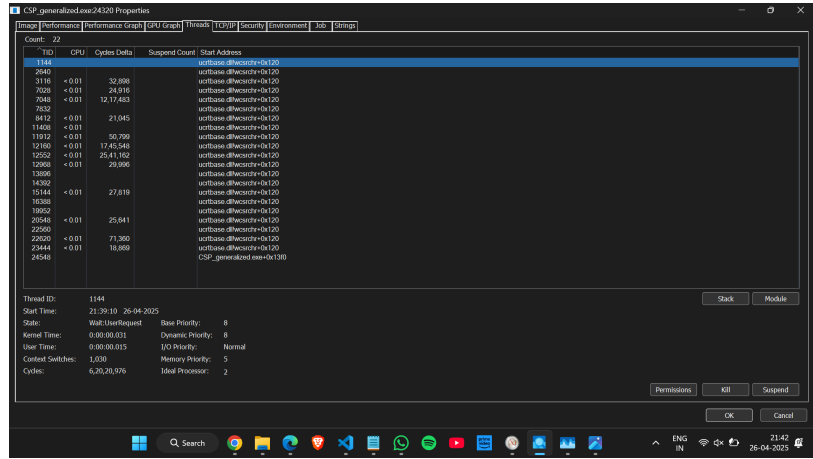


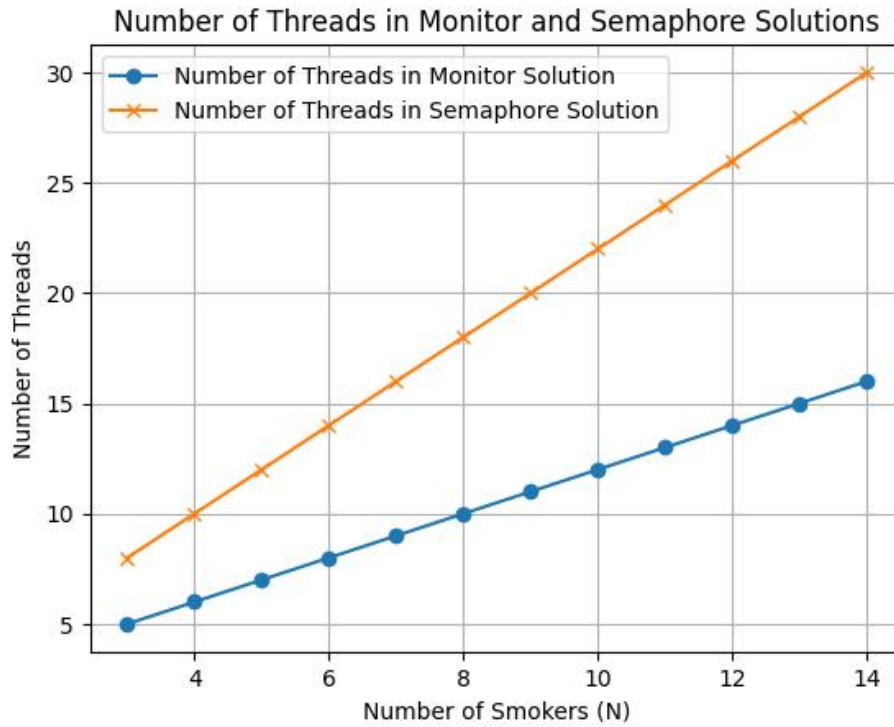
Figure 9: Context Switches of Main Thread in the Semaphore Solution

The number of context switches of the agent thread and the smoker thread has been consistent in both the solutions (semaphores and monitors).

The difference between the solutions of semaphores and monitors can be summarised in the table given below:

Metric	Semaphore Solution	Monitor Solution
Memory Used	2,00,000 KB	150 KB
Page Faults	70,000	1500
Throughput (Smoking Rounds per second)	30,000	90,000
Number of threads formed	22	12
Average number of context switches (agent thread)	1000	1000

Table 3: Comparison Between Semaphore and Monitor Solutions for $N = 10$ smokers



5 Conclusion

In this report, we explored the Cigarette Smokers' Problem and the issue with naive usage of semaphores. Thus, we realised that the usage synchronisation tools must be well-thought out. While the cigarette smokers' problem is a hypothetical scenario, it models different processes in the same system that are trying to access the same resources. The conditions put by Patil in the problem statement are highly practical as the problem is supposed to model an operating system scenario. In a system, the processes competing for the same resources are not supposed to be aware of each other, and the resource distribution is seemingly arbitrary, but still needs to avoid deadlock. Thus, the learnings of our project are directly applicable to real scenarios.

Through our extensive comparative analysis of semaphores and monitors, we realised the advantages and disadvantages of both. We already knew that semaphores are lower-level synchronisation tools.

This means that they are easier to deal with from the perspective of the developer, in terms of code complexity, but are prone to errors. Monitors abstract the components providing greater safety.

Specifically, through the analysis, we observe from this report that monitors can be considered better than semaphores in solving the cigarette smokers' problem as the solution with monitor has better CPU throughput as well as lesser memory usage.

5.1 Challenges Faced

Right from the beginning, we faced challenges in understanding Patil's paper as it involved highly complex structures, such as Petri nets. We had to read assisting papers to understand the concepts. After understanding the problem statement, coming up with the solution was also a very thought-provoking task.

The implementation was also challenging, particularly the semaphore solution. The naive implementation of semaphores used busy waiting, which was highly inefficient. We realised that our solutions were correct but not providing the desired results due to the 'hold and wait' implementation. We had to modify our source code and it made us realise the importance of understanding the primitives and their usage.

We built on our knowledge of semaphores through the project and then implemented monitors. The monitor solution required an altogether different approach and we had to refine our understanding about mutex locks and conditional variables.

References

- [1] Suhas Patil, *Limitations and Capabilities of Dijkstra's Semaphore Primitives for Coordination among Processes*, Project MAC, Massachusetts, February 1971.
- [2] Nico Habermann, *On a Solution and a Generalization of the Cigarette Smokers' Problem*, August 1972.
- [3] D.L. Parnas, *On a Solution to the Cigarette Smokers' Problem (without conditional statements)*, Technische Hochschule Darmstadt, March 1975.
- [4] Avi Silberschatz, Peter B Galvin, Greg Gagne, *Operating System Concepts*, 2019.
- [5] Allen Downey, *Little Book of Semaphores*, 2016.

We would like to sincerely thank Prof. Peddoju Sateesh Kumar for teaching us the topic and providing valuable insights throughout the course. His guidance and expertise have been instrumental in our understanding of the subject as well as the completion of this project.